

Unit Tests

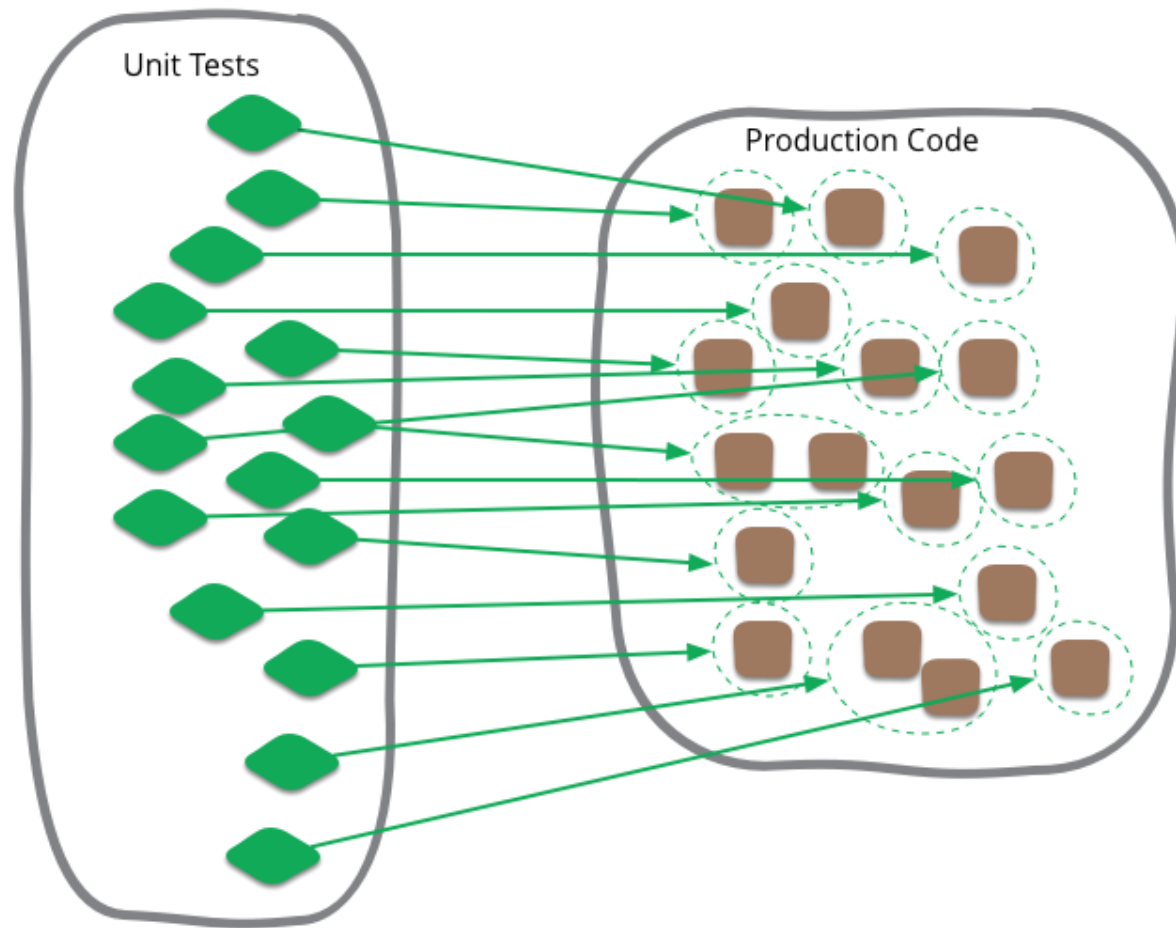
Visual Studio 2017

Einführung und Überblick

Testgetriebene Entwicklung [I]

- Ziel
 - Reduzieren von Bugs
 - Steigerung der Qualität der Software
- Ansatz
 - Reihenfolge: Zuerst Tests dann Algorithmus
 - Abdeckung des gesamten Programms (Code Coverage)
 - Implementierung der Tests optimal durch anderen Programmierer
 - Automatische Ausführung der Tests (Werkzeugunterstützung)

Testgetriebene Entwicklung [II]



Neue Solution erstellen



Hülle für Algorithmus (Programm) und Testprojekt

1. File > New > Project

2. Visual Studio Solutions > „Blank Solution“

3. Namen vergeben

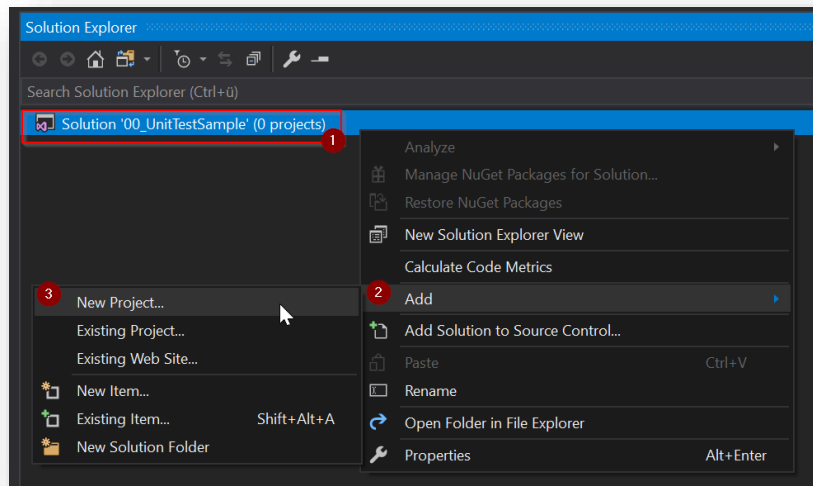
4. Pfad wählen

5. „OK“

Neues Projekt erstellen [I]



Projekt für Algorithmus

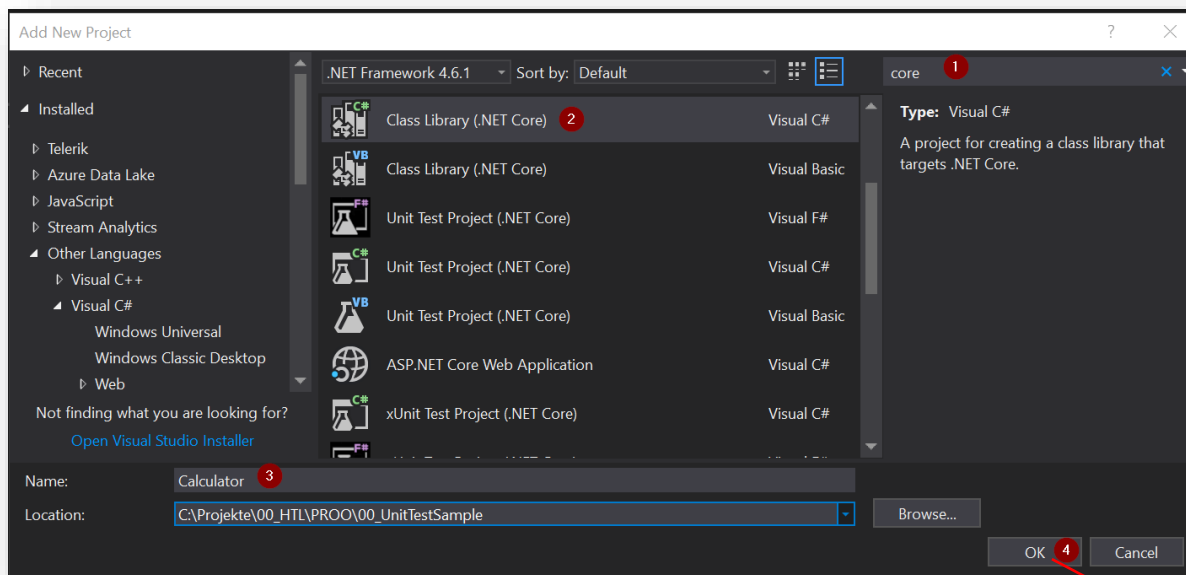


1. Solution > Kontextmenü (1)
2. Add (2) > New Project (3)

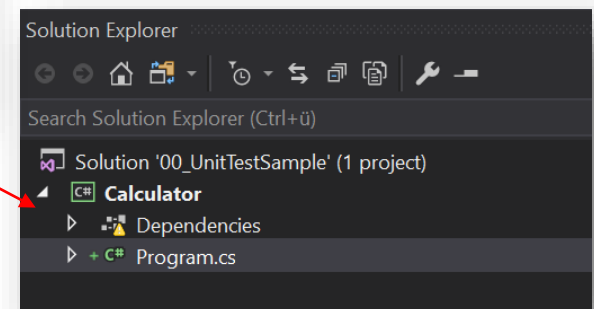
Neues Projekt erstellen [II]



Projekt für Algorithmus



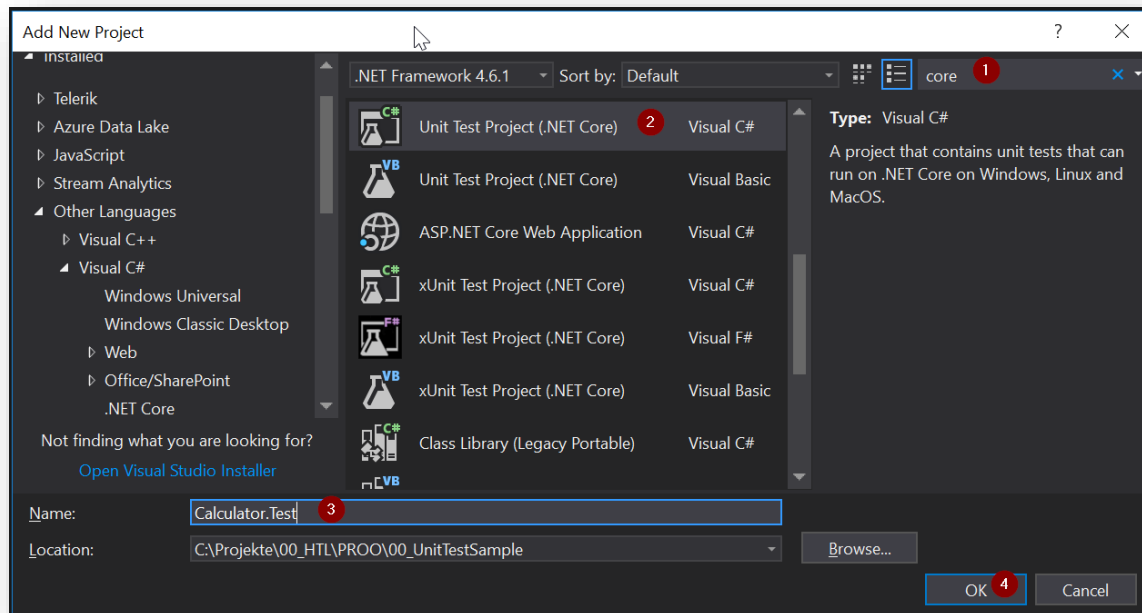
1. Filter „core“ (1)
2. „**Class Library (.NET Core)**“ (2)
3. Namen vergeben (3)
4. OK (4)



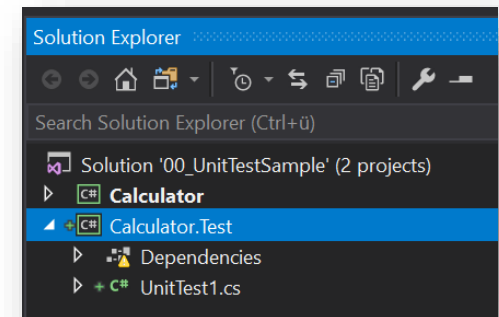
Neues Projekt erstellen [III]



Projekt für Tests



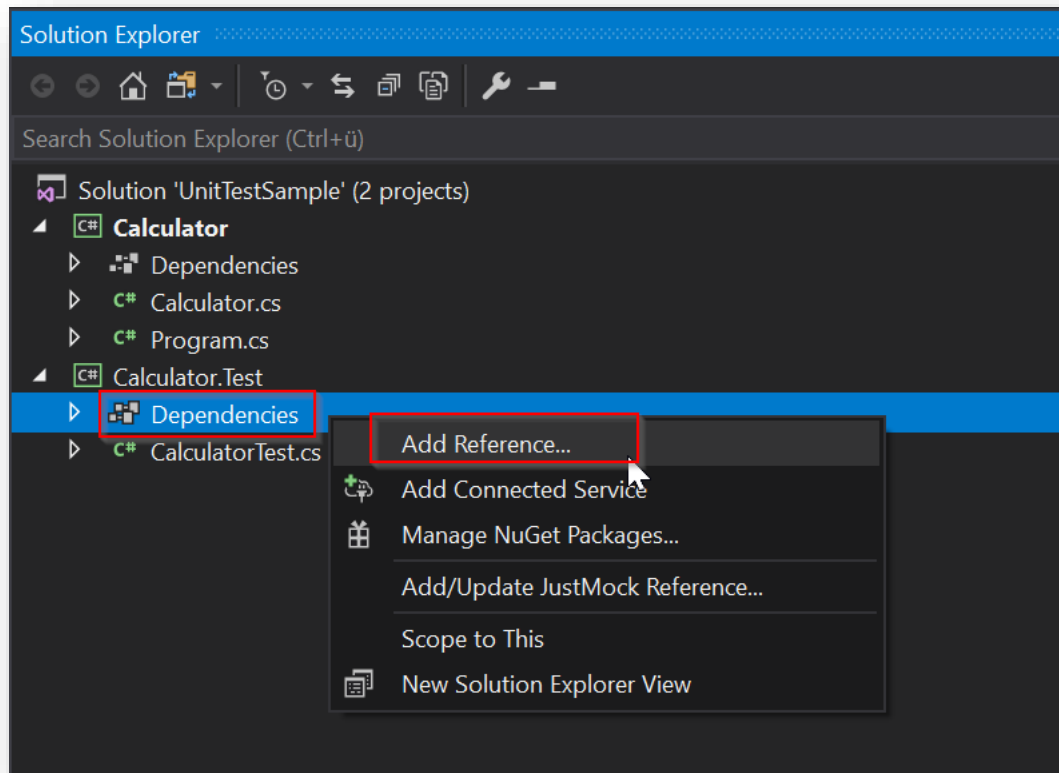
1. Filter „core“ (1)
2. „Unit Test Project (.NET Core)“ (2)
3. Namen vergeben (3)
4. OK (4)



Referenz setzen [I]



Testprojekt kennt Algorithmus

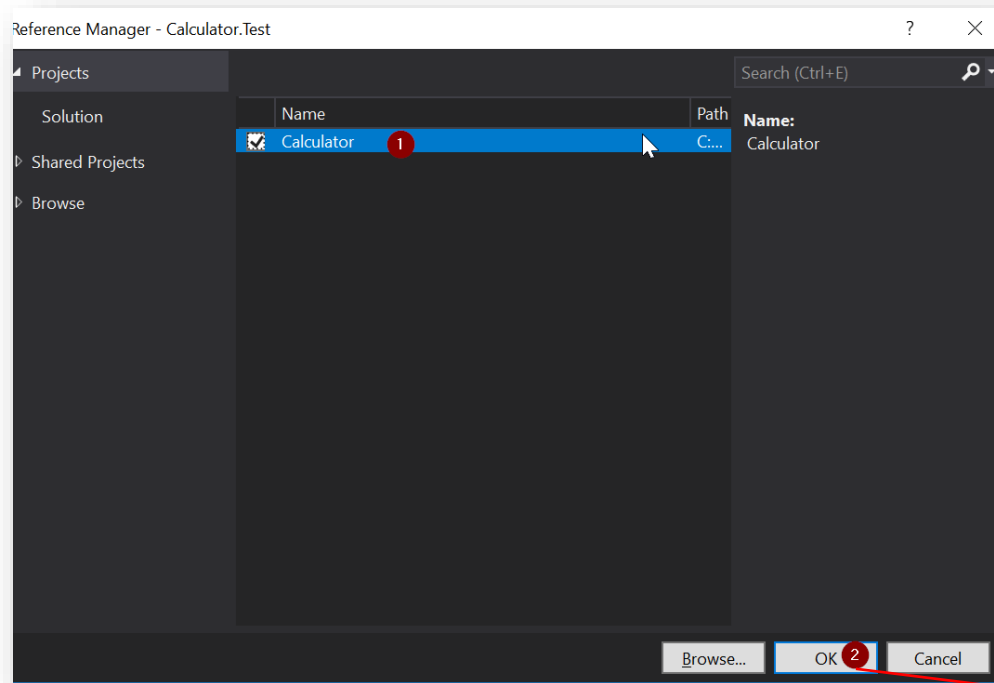


1. Testprojekt > Kontextmenü
2. Add Reference

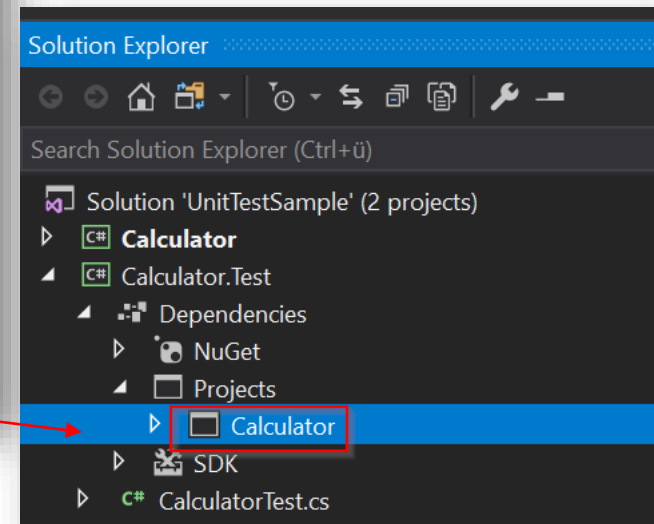
Referenz setzen [II]



Testprojekt kennt Algorithmus



1. Projekt mit Algorithmus auswählen (1)
2. OK (2)



Algorithmus-Methoden erstellen



Tests können relevante Methoden des Algorithmus aufrufen

```
Calculator.cs  + X
Calculator
1  using System;
2
3  namespace Calculator
4  {
5      2 references | 0 changes | 0 authors, 0 changes
6      public class Calculator
7      {
8          /// <summary>
9          /// Adds two numbers.
10         /// </summary>
11         /// <param name="a">Holds first operand.</param>
12         /// <param name="b">Holds second operand.</param>
13         /// <returns>The result of the operation.</returns>
14         1 reference | 1/1 passing | 0 changes | 0 authors, 0 changes
15         public static int Add(int a, int b)
16         {
17             throw new NotImplementedException();
18         }
19     }
```

- Relevante Methoden => public
- Initial: Werfen von Fehlern „NotImplementedException“
- Keine Kompilierfehler!

Test-Methoden erstellen

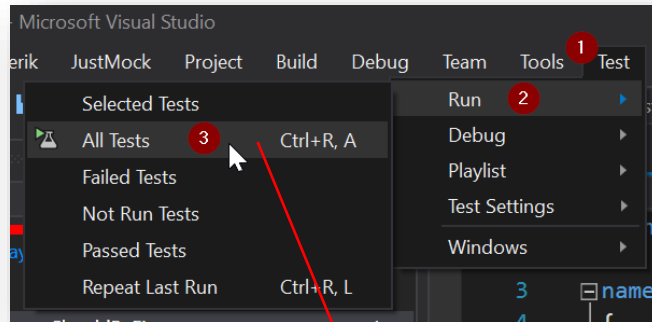


Tests rufen relevante Methoden des Algorithmus auf

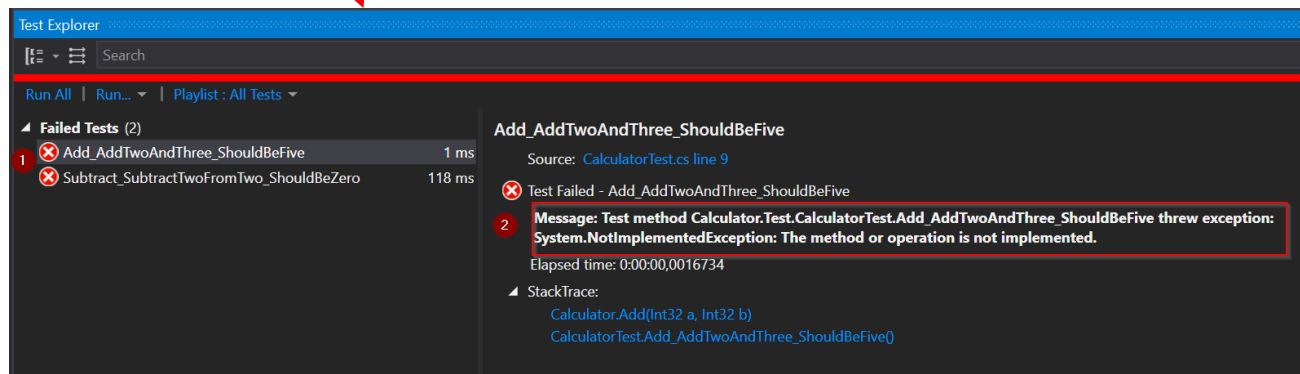
```
CalculatorTest.cs x
Calculator.Test
1  using Microsoft.VisualStudio.TestTools.UnitTesting;
2
3  namespace Calculator.Test
4  {
5      1 [TestClass]
        0 references | 0 changes | 0 authors, 0 changes
6      public class CalculatorTest
7      {
8          2 [TestMethod]
        0 references | 0 changes | 0 authors, 0 changes
9          public void Add_AddTwoAndThree_ShouldBeFive()
10         {
11             3 Assert.IsTrue(Calculator.Add(2, 3) == 5);
12         }
13     }
```

- Klasse ist mit „[TestClass]“ dekoriert (1)
- Test ist mit „[TestMethod]“ dekoriert (2)
- Test prüft per Assert.IsXXX() (3)

Tests im TestExplorer laufen lassen



Test > Run > All Tests [Ctrl+R, A]



Alle Tests schlagen fehl (1)

Grund: „NotImplementedException“ (2)

Algorithmus implementieren + Retest

```
/// <summary>
/// Adds two numbers.
/// </summary>
/// <param name="a">Holds first operand.</param>
/// <param name="b">Holds second operand.</param>
/// <returns>The result of the operation.</returns>
1 reference | 1/1 passing | 0 changes | 0 authors, 0 changes
public static int Add(int a, int b)
{
    return a + b; 1
}

/// <summary>
/// Subtracts two numbers.
```

Quick Actions and Refactorings...	Ctrl+.
Rename...	Ctrl+R, Ctrl+R
Remove and Sort Usings	Ctrl+R, Ctrl+G
Peek Definition	Alt+F12
Go To Definition	F12
Go To Implementation	Ctrl+F12
Find All References	
View Call Hierarchy	Ctrl+K, Ctrl+T
Run Tests	Ctrl+R, T
Debug Tests	Ctrl+R, Ctrl+T
Breakpoint	

1. Implementierung abschließen
2. Run Tests
3. **Alle Tests müssen erfolgreich sein**

